

DRAC: Direction-aware Reconfigurable Asymmetric Connectivity for Large-Scale AI Training

HPCA 2027 Submission #20

Confidential Draft

Do NOT Distribute!!

Abstract—Current AI networks pay an $xx\%$ premium for reverse-direction capacity that is rarely, if ever, used. In contrast, the deterministic traffic patterns of LLM training make the demand in each direction precisely predictable. OCS technology, for the first time, makes it feasible to build a network architecture that provisions capacity according to actual traffic demand. This capability marks a paradigm shift from general-purpose symmetric networks to AI-native asymmetric networks.

I. INTRODUCTION

The relentless scaling of large language models (LLM) has made datacenter networks a first-order bottleneck for large-scale training [1]–[3]. As GPU clusters grow to thousands of devices, collective communication increasingly dominates end-to-end training time [9], [15], [16]. Existing AI fabrics are almost universally provisioned with symmetric bidirectional capacity: the links between any pair of network endpoints carry identical bandwidth in both directions. **This design is simple and general-purpose [20], [21], yet it implicitly assumes balanced traffic demand in both directions of every path.**

In practice, this assumption breaks down for the dominant communication domains in LLM training. Although collectives such as all-gather and reduce-scatter are logically symmetric [13], [15], the induced point-to-point traffic between ordered network endpoints is fundamentally asymmetric. In Data Parallelism, one direction carries tensor or gradient payloads, while the reverse transmits only lightweight control traffic. Our measurements (Figure 2) show that DP gradient synchronization exhibits a transmitted-to-received ratio of up to **141x** per iteration; the forward path is payload-dominated, whereas the reverse consists mainly of RoCEv2 control packets and CTS messages (Figure 3). Under port-limited fabrics, this mismatch wastes scarce resources on low-demand reverse directions while starving high-demand forward paths. **We argue that network connectivity should be provisioned asymmetrically according to actual directional demand.**

Optical Circuit Switching (OCS) uniquely enables this because its **input and output ports are physically decoupled and independently matchable**, whereas Electrical Packet Switching (EPS) has bidirectional ports that are bound at the transceiver and switch-silicon level. **In OCS, a bidirectional port pair consists of two independent unidirectional fibers, each matchable to any opposite port.** By rematching these ports, OCS can assign different numbers of logical connection units to the two ordered directions of a node pair without

altering physical link rates or collective semantics. Existing OCS-based training networks [4], [6], [10], [18], [22] reallocate bandwidth among communication domains but they do not consider the asymmetric configuration of capacity between node pairs; topology or scheduling optimizations [11], [17], [19], [21], [23] operate above the connectivity layer and do not realize directional demand at ordered path granularity. **To the best of our knowledge, we are the first to exploit OCS reconfigurability for direction-aware asymmetric connectivity in LLM training fabrics.**

Realizing this vision confronts three structural tensions that arise specifically from direction-aware provisioning. Prior OCS systems [4], [22] address generic discrete allocation and phase-level reconfiguration, but not the unique conflicts created by asymmetric demand.

1. Recovering ordered demand. Collective operations expose only symmetric APIs; the ordered payload distribution is buried inside NCCL [16] and interleaved across TP/DP/PP phases. Without modifying the training stack, how do we deterministically recover the ordered node-pair traffic matrix?

2. Realizing continuous directional targets with discrete optical resources. The desired capacity of each ordered direction is continuous, whereas OCS connectivity is provided in indivisible, fixed-bandwidth units. For example, the logical target may require 2.7 units $A \rightarrow B$ but only 0.3 units $B \rightarrow A$. These two directions cannot be rounded independently; assigning one additional $A \rightarrow B$ connection consumes one Tx channel at A and one Rx channel at B, and vice versa. Moreover, all ordered pairs compete for the same per-server Tx and Rx resources and must jointly form a feasible OCS matching. Naive rounding may therefore violate resource constraints or erase the intended directional bias. How can the continuous target be converted into a feasible integer configuration that closely preserves its asymmetry while accurately identifying the optical channels left available for other jobs?

3. Structural reconfiguration vs. temporal demand flipping. Training workloads follow a static, repeating phase sequence (e.g., TP all-gather, DP gradient sync, PP forward/backward). Each phase induces a distinct ordered traffic matrix, and reconfiguring OCS circuits between phases incurs non-negligible switching latency. Frequent reconfiguration is prohibitive, yet a static asymmetric topology applied to flipped demand wastes capacity on the wrong direction. How do we segment the sequence into epochs where a single asymmetric structure remains effective?

To resolve these tensions, we present **DRAC** (Direction-aware Reconfigurable Asymmetric Connectivity), a cross-layer framework built on three key insights.

Insight 1: Static graph deconstruction. DRAC’s Demand Profiler parses the static training computation graph to map logical collectives onto ordered node-pair flows, exposing directional skew without instrumenting NCCL or altering the training stack. This addresses **Challenge 1**.

Insight 2: Feasibility-preserving directional quantization and peak-residual resource exposure. The **Logical Target Generator** computes a continuous directional bandwidth allocation that minimizes the maximum completion time across ordered directions, generalizing water-filling to asymmetric objects. The **OCS Realization Engine** then resolves the coupled packing problem through conservative floor rounding to guarantee strict per-node port feasibility, followed by greedy residual-gap filling to preserve directional bias. The **Physical Resource Exposure Planner** computes peak outbound and inbound usage across all segments, marking only sub-peak resources as releasable; for bidirectional transceivers, it adopts a conservative max-outbound/inbound anchoring policy. This separates feasibility (satisfying coupled budgets) from optimality (preserving skew) and from deployability (safely exposing trapped resources), avoiding intractable integer programming. This addresses **Challenge 2**.

Insight 3: Offline segmentation for directional stability. The **Logical Target Generator** uses dynamic programming to partition the training communication sequence into segments sharing a single asymmetric configuration, explicitly modeling OCS reconfiguration overhead in the recurrence. By treating the directional bandwidth matrix as the allocation object—rather than a scalar domain bandwidth—it finds the Pareto-optimal trade-off between structural inertia and the cost of rotating the skew pattern across phases. Because the segmentation is computed entirely offline, runtime overhead is eliminated. This addresses **Challenge 3**.

Together, these layers form an end-to-end pipeline: the logical layer determines what directional capacity is needed and when the structure should change; the realization layer determines how to instantiate it with discrete OCS ports under coupled constraints; and the physical layer determines which resources can be safely redeployed. DRAC operates entirely below the collective layer.

We implement DRAC as a topology-agnostic control plane that parses the static training computation graph, maps lower-level endpoints to abstract OCS-attached nodes via a configurable mapping function, and emits a sequence of integer OCS connection matrices. Each matrix specifies the exact number of directional connection units between abstract node pairs for a communication segment, and is directly consumable by existing OCS controllers or SDN planes without modifying NCCL or the training stack.

We evaluate DRAC on LLaMA-3 [5] workloads spanning **XX, XX, XX, and XX**. Compared to static symmetric fabrics and OCS-enabled symmetric baselines (ACTINA-style), DRAC reduces communication time by up to **xx%** for DP

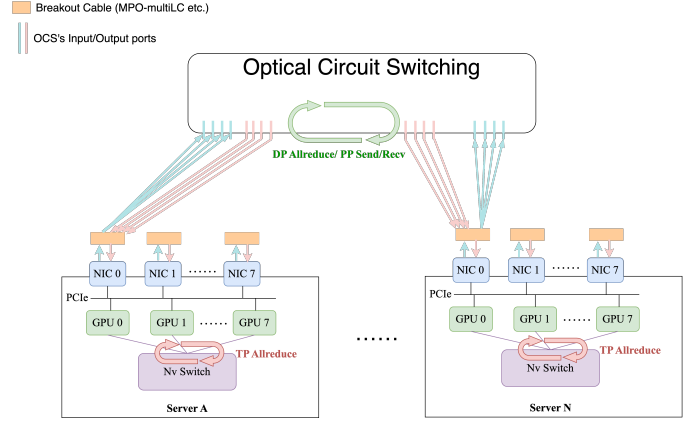


Fig. 1. System architecture of an OCS-enabled multi-GPU training cluster. Each server consists of multiple GPU–NIC pairs connected through PCIe, while intra-server TP communication is handled by NVSwitch. Cross-server DP and PP communication traverse the NIC-attached optical network. Each NIC connects to the OCS through a breakout optical interface that exposes multiple Tx/Rx optical channels as independent optical resources.

under strong skew, while reducing symmetric capacity waste across all port budgets. DRAC matches the performance of a xx -port-per-node symmetric OCS fabric using only xx ports for xx and xx ports for DP, freeing resources for other tasks. Under near-symmetric PP traffic, DRAC remains within $xx\%$ of the symmetric baseline, confirming it does not force harmful asymmetry where it is not beneficial.

II. BACKGROUND

This section introduces the communication and optical-network foundations of DRAC. We first define the server boundary considered in this work, then examine the phase-level directional characteristics of DP and PP communication. We finally explain how independently configurable Tx and Rx optical channels enable directional capacity provisioning and illustrate why symmetric provisioning can mismatch ordered communication demand.

A. Inter-Server Communication in Multi-GPU Training

Modern LLM training servers integrate multiple GPUs, high-speed NICs, and a high-bandwidth intra-server interconnect. A representative server contains N GPUs and N NICs, with $N = 8$ being a common configuration, and each GPU is associated with a NIC-facing network path. The GPUs within a server communicate through NVLink and NVSwitch, which provide substantially higher local communication bandwidth than the inter-server network [9], [14], [15].

Training systems commonly combine Tensor Parallelism (TP), Data Parallelism (DP), and Pipeline Parallelism (PP). In the deployment considered by DRAC, TP groups are primarily contained within individual servers, so TP collective communication is carried by the intra-server NVSwitch fabric. In contrast, DP and PP communication may span multiple servers, As shown in figure 1.

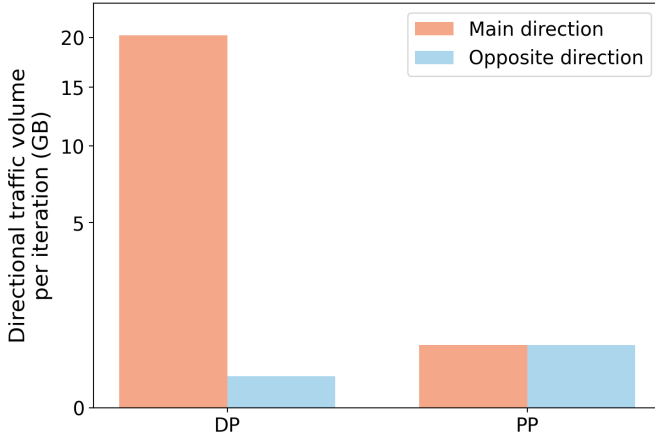


Fig. 2. Derived directional communication demand of representative DP and PP phases. For each communication pattern, the main direction denotes the higher-volume ordered direction between two GPU–NIC endpoints, while the opposite direction denotes the reverse direction within the same communication interval. DP synchronization exhibits a strong directional skew due to payload-dominant gradient propagation, whereas PP becomes near-balanced when forward activations and backward gradients are aggregated over a full iteration. The y-axis uses a square-root scale for readability.

DRAC focuses on these inter-server communications. The server remains the physical deployment boundary, but the communication endpoint is associated with each GPU–NIC pair rather than an abstract server port. Each NIC exposes optical resources to the network through breakout optical interfaces, and these resources are configured by the OCS. DRAC does not assume GPU-to-NIC pooling, in-server traffic relaying, or changes to model placement and collective semantics. Instead, it operates on the native communication paths provided by modern GPU clusters.

B. Directional Characteristics of Training Communication

Inter-server communication in LLM training can exhibit different directional characteristics across training phases. Although collective operations expose symmetric communication semantics, the underlying point-to-point traffic between GPU–NIC endpoints may become directional depending on the communication schedule and the aggregation interval. DP collectives may generate payload-dominant flows along specific ordered endpoint directions, whereas PP transfers activations and gradients in opposite directions during forward and backward propagation. Therefore, whether a pair of endpoints requires symmetric or asymmetric connectivity depends on the concrete communication phase.

Figure 2 illustrates this difference using representative DP and PP phases. DP synchronization exhibits a strong directional skew, where one ordered direction carries substantially more traffic than the reverse direction. In contrast, when PP communication is aggregated over a full iteration, forward activation transfers and backward gradient transfers result in comparable demands in the two directions. This observation motivates a connectivity model that does not enforce symmetry

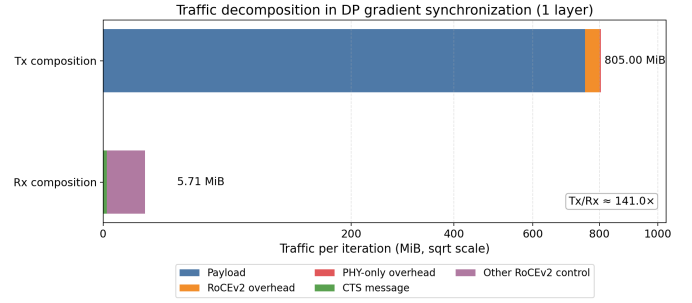


Fig. 3. Traffic decomposition of DP gradient synchronization for one layer, measured from actual NCCL point-to-point communication executions. The x-axis uses a square-root scale for readability.

or asymmetry by design, but instead allows the optical fabric configuration to follow the communication demand.

1) *Data Parallelism*: DP synchronizes gradients among model replicas, commonly through NCCL Ring AllReduce. In a ring topology, each GPU receives gradient chunks from its predecessor and forwards them to its successor, forming a unidirectional communication ring [7], [16]. Thus, for two adjacent GPU–NIC endpoints, one direction carries the bulk gradient payload, while the reverse direction mainly carries lightweight transport-control traffic.

Figure 3 further decomposes the traffic of DP gradient synchronization based on actual NCCL point-to-point communication executions. The result shows that the dominant direction is primarily composed of gradient payload and its associated transport overhead, whereas the opposite direction contains only a small amount of control traffic. Note that this pairwise directionality does not imply that an individual GPU has imbalanced aggregate transmission and reception volumes. A GPU may receive data from one neighbor while sending comparable data to another neighbor. The imbalance appears between the two ordered directions of a particular endpoint pair, which is the granularity considered by DRAC.

2) *Pipeline Parallelism*: PP transfers activations in the forward direction and gradients in the reverse direction between adjacent pipeline stages [8], [12], [15]. When a configuration interval covers both forward and backward propagation, the two ordered directions may carry more comparable traffic than in DP, the exact balance depends on the microbatch schedule, tensor sizes, stage placement, and the length of the configuration interval.

The balance of PP traffic further depends on factors including microbatch scheduling, tensor sizes, stage placement, and the duration of an OCS configuration interval. Therefore, DRAC does not assume that PP communication is inherently symmetric. Instead, PP represents an important case where a direction-aware fabric should naturally converge to a symmetric configuration when the two ordered directions carry similar demand.

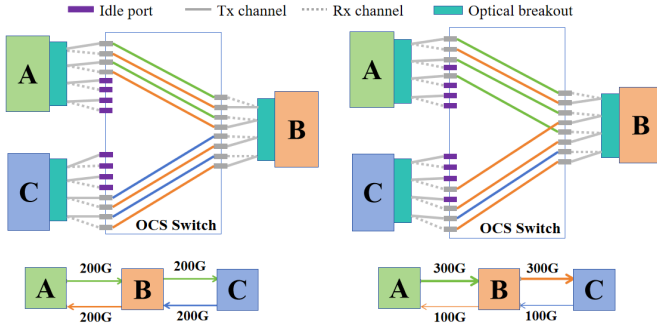


Fig. 4. OCS-enabled directional optical connectivity. To keep the port-level mapping legible, endpoints A, B, and C are shown as an unfolded segment of a communication ring; the omitted closing edge follows the same directional matching mechanism and does not alter the allocation model.

C. OCS-Enabled Directional Optical Connectivity

The directional traffic patterns described above can be supported by an OCS-enabled optical fabric. As illustrated in Figure 4, each GPU–NIC endpoint connects to the optical fabric through an optical breakout that exposes multiple fixed-rate unidirectional channels. The OCS establishes a directed optical connection by matching one source Tx channel with one destination Rx channel. Multiple such connections between the same ordered endpoint pair can be aggregated to provide higher directional capacity.

The left side of Figure 4 shows a symmetric configuration. For each adjacent endpoint pair, the OCS establishes the same number of connections in the two ordered directions, providing 200 Gbps in each direction. This configuration does not distinguish whether one direction carries substantially more traffic than its reverse. The right side shows a directional configuration under the same aggregate connection budget. The OCS assigns three connection units to the dominant direction and one unit to the reverse direction, providing 300 Gbps and 100 Gbps, respectively.

Importantly, directional provisioning does not convert Rx channels into Tx channels. Instead, it realizes directional capacity by selecting different numbers of feasible source–Tx–destination–Rx matches for the two ordered directions. Consider a ring in which every endpoint exposes $2n$ Tx channels and $2n$ Rx channels ($2n = 4$ in Fig 4). Under symmetric provisioning, endpoint i connects n Tx channels to its successor and n Tx channels to its predecessor, while receiving n channels from each neighbor. To bias capacity toward one ring direction, the OCS can instead connect $n + \Delta$ Tx channels from every endpoint to its successor and $n - \Delta$ Tx channels to its predecessor ($\Delta = 1$ in Fig 4). The corresponding incoming connections are formed consistently around the ring. Consequently, each endpoint still uses exactly

$$(n + \Delta) + (n - \Delta) = 2n$$

Tx channels and the same number of Rx channels. If the smaller directional allocation is denoted by $m = n - \Delta$, the

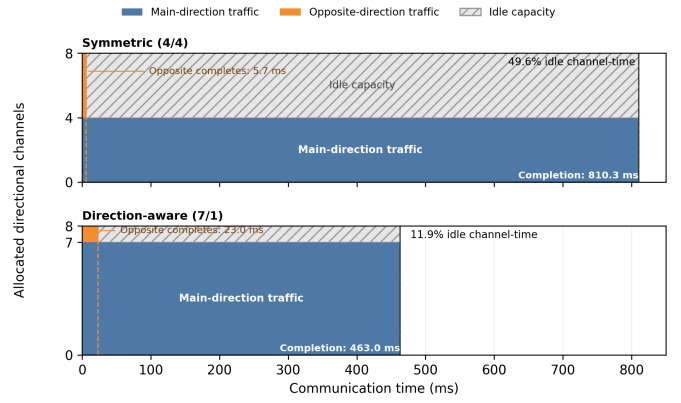


Fig. 5. Channel-time utilization under a representative directional D demand. Both configurations share a budget of eight equal-rate unidirectional OCS connection units.

per-edge split can equivalently be written as $(2n - m)/m$. For example, a symmetric 4/4 configuration becomes 7/1 by setting $n = 4$ and $\Delta = 3$ (or $m = 1$). No optical channel changes its Tx or Rx role; the OCS only changes which destination Rx channel is matched to each existing source Tx channel.

For clarity, Figure 4 depicts endpoints A, B, and C as an unfolded segment rather than drawing a closed communication ring. In an actual ring collective, the final endpoint is also connected back to the first endpoint, and the same directional matching mechanism applies to that closing edge. Explicitly drawing the closing connection in the port-level view would introduce several crossing optical paths. The unfolded representation therefore simplifies visualization without changing the underlying ring communication pattern or DRAC’s per-ordered-pair allocation model.

This capability makes symmetric connectivity a special case of a more general directional optical configuration. When the two ordered demands are comparable, the OCS can establish equal capacity in both directions. When one direction dominates, it can assign more connection units to that direction while retaining sufficient reverse capacity. The following motivating example quantifies the performance cost of retaining the symmetry constraint under a strongly directional DP demand.

D. Motivating Example: The Cost of Symmetric Provisioning

We use a simple micro-simulation to illustrate how enforcing symmetric connectivity can translate directional traffic skew into longer communication time. The experiment uses the representative ordered DP demand shown in Figure 2. We consider a shared budget of eight equal-rate unidirectional OCS connection units assignable to this endpoint pair, each providing 50 Gbps. The two provisioning strategies therefore use the same aggregate connection budget and the same per-unit line rate.

Under this eight-unit budget, the best feasible symmetric configuration assigns four units to each ordered direction.

As shown in Figure 5, the four units assigned to the main direction require 810.3 ms to deliver the payload-dominant traffic. In contrast, the opposite-direction traffic completes in only 5.7 ms. Its four assigned units consequently remain idle for 99.3% of the communication interval while the main direction continues transmitting. This idle reverse capacity accounts for 49.6% of the total available channel-time.

A direction-aware configuration instead selects a 7/1 allocation. The seven main-direction units reduce the payload completion time to 463.0 ms, while the remaining opposite-direction unit completes its traffic in 23.0 ms. The resulting total idle channel-time fraction decreases to 11.9%, and the synchronization interval becomes 42.9% shorter than under symmetric provisioning.

The improvement does not result from adding optical resources or increasing the line rate. It comes from assigning the existing connection units according to the two ordered demands, rather than forcing the same capacity in both directions. Conversely, when the two directions carry comparable traffic, the completion-time-minimizing directional allocation naturally becomes symmetric. Symmetric connectivity is therefore a special case of directional provisioning, rather than a constraint that should be imposed on every communication phase. We provide a broader sensitivity analysis across different directional traffic ratios in Appendix IX-A

III. PROBLEM FORMULATION

We consider an OCS-enabled multi-GPU training fabric in which GPU–NIC endpoints expose a limited number of transmit and receive optical channels. A recurring training iteration contains a sequence of communication nodes extracted from the training computation graph. Each communication node induces ordered traffic demands between GPU–NIC endpoints. Because the two directions of an endpoint pair may carry different traffic volumes, they are modeled as independent provisioning objects.

The problem is to partition the communication-node sequence into contiguous configuration segments and assign one feasible integer OCS configuration to each segment. The objective is to minimize communication completion time while accounting for OCS reconfiguration overhead. This section defines the demand model, configuration constraints, and optimization objective. Section IV presents DRAC’s solution to this problem.

A. System Model

Let $\mathcal{U}$ denote the set of GPU–NIC endpoints participating in a training job. Each endpoint $u \in \mathcal{U}$ belongs to a physical server, denoted by $s(u)$. Communication between endpoints on the same server uses the intra-server fabric and does not consume the OCS-facing resources considered in this work.

The training computation graph induces a communication subgraph

$$\mathcal{G}_c = (\mathcal{P}, \mathcal{E}_c),$$

where

$$\mathcal{P} = (p_1, p_2, \dots, p_K)$$

is the ordered sequence of communication nodes under the static training schedule. Each node p_k represents one communication operation, such as a collective operation or a point-to-point pipeline transfer.

Each communication node p_k induces an ordered demand matrix

$$\mathbf{T}^{(k)} = \left\{ T_{uv}^{(k)} \right\}_{u,v \in \mathcal{U}},$$

where $T_{uv}^{(k)}$ denotes the traffic volume transmitted from endpoint u to endpoint v by communication node p_k . We set

$$T_{uv}^{(k)} = 0 \quad \text{if } u = v \text{ or } s(u) = s(v),$$

because such traffic does not traverse the inter-server optical fabric.

The demand matrix is ordered. In general,

$$T_{uv}^{(k)} \neq T_{vu}^{(k)}.$$

For example, a DP communication node may carry gradient payloads in one direction while the reverse direction carries only lightweight transport-control traffic. PP forward and backward nodes may favor opposite directions; when they are grouped into the same configuration segment, their combined requirements may become closer to symmetric.

We allow an endpoint pair to have an existing baseline capacity B_{uv}^{fix} . This capacity may be provided by statically reserved optical connectivity or another always-available path. It is not controlled by the directional provisioning decision. A deployment without such a baseline is represented by setting $B_{uv}^{\text{fix}} = 0$.

Each endpoint u exposes n_u^{tx} transmit channels and n_u^{rx} receive channels that can be configured through the OCS. Each optical connection unit provides a fixed bandwidth c .

B. Feasible Directional Configuration

Let

$$X_{uv} \in \mathbb{Z}_{\geq 0}$$

denote the number of reconfigurable optical connection units assigned from endpoint u to endpoint v in one OCS configuration.

A connection from u to v consumes one transmit channel at u and one receive channel at v . Therefore, a feasible configuration must satisfy

$$\sum_{v \neq u} X_{uv} \leq n_u^{\text{tx}}, \quad \forall u \in \mathcal{U}, \quad (1)$$

$$\sum_{v \neq u} X_{vu} \leq n_u^{\text{rx}}, \quad \forall u \in \mathcal{U}. \quad (2)$$

Since intra-server communication does not use the OCS-facing fabric, we further require

$$X_{uv} = 0, \quad \forall s(u) = s(v). \quad (3)$$

We use $\mathcal{F}_{\text{OCS}}$ to denote the set of integer matrices satisfying Equations 1–3. Under configuration $\mathbf{X}$, the realized capacity of ordered direction $u \rightarrow v$ is

$$\hat{B}_{uv}(\mathbf{X}) = B_{uv}^{\text{fix}} + cX_{uv}.$$

The formulation does not require $X_{uv} = X_{vu}$. The two directions consume different physical resources: $u \rightarrow v$ uses a Tx channel at u and an Rx channel at v , and vice versa. A symmetric OCS system can be represented as a restricted version of the same feasible set with the additional constraint

$$X_{uv} = X_{vu}, \quad \forall u, v \in \mathcal{U}.$$

Thus, symmetric connectivity is a special case of the more general directional configuration problem.

C. Communication and Reconfiguration Cost

For a fixed OCS configuration $\mathbf{X}$, the completion time of communication node p_k is modeled as

$$L_k(\mathbf{X}) = \max_{u \neq v} \frac{T_{uv}^{(k)}}{\widehat{B}_{uv}(\mathbf{X})}. \quad (4)$$

This model captures the fact that a communication node cannot complete before its slowest ordered transfer finishes. If $T_{uv}^{(k)} > 0$ but $\widehat{B}_{uv}(\mathbf{X}) = 0$, the corresponding ratio is defined as infinity.

Using a different OCS configuration for every communication node may closely track instantaneous demand, but would incur frequent optical reconfiguration. We therefore group consecutive communication nodes into configuration segments. Let $0 = q_0 < q_1 < \dots < q_M = K$ denote the segment boundaries, where M is the number of configuration segments. The m -th segment is the contiguous subsequence

$$\mathcal{I}_m = \{p_{q_{m-1}+1}, p_{q_{m-1}+2}, \dots, p_{q_m}\}, \quad 1 \leq m \leq M.$$

These boundaries partition the communication-node sequence into non-overlapping and contiguous segments. Every communication node belongs to exactly one segment, and the original execution order is preserved.

All communication intervals in $\mathcal{I}_m$ share one integer OCS configuration $\mathbf{X}^{(m)}$. The communication cost of segment m is therefore

$$C_m(\mathbf{X}) = \sum_{k=q_{m-1}+1}^{q_m} L_k(\mathbf{X}).$$

Let δ denote the exposed latency of changing from one OCS configuration to another. Given the segment boundaries and their configurations, the total execution cost is

$$C_{\text{total}} = \sum_{m=1}^M \left[C_m(\mathbf{X}^{(m)}) + \mathbb{I}[m > 1]\delta \right], \quad (5)$$

where $\mathbb{I}[\cdot]$ is the indicator function. The reconfiguration cost is charged only when execution moves between adjacent segments. The first configuration is installed before the recurring communication sequence begins and is therefore excluded from the steady-state reconfiguration cost.

Equation 5 captures the trade-off between demand matching and configuration stability. More segments allow the optical connectivity to follow changes in node-level directional demand more closely, but introduce additional reconfiguration overhead. Fewer segments reduce this overhead, but force one configuration to serve communication nodes that may have different directional requirements.

D. Directional Connectivity Provisioning

Let $\mathcal{Q} = \{q_0, q_1, \dots, q_M\}$ denote a valid segmentation of the communication-node sequence. The directional connectivity provisioning problem selects both the segment boundaries and one feasible integer OCS configuration for each segment:

$$\min_{\mathcal{Q}, \{\mathbf{X}^{(m)}\}} C_{\text{total}}, \quad (6)$$

subject to

$$\mathbf{X}^{(m)} \in \mathcal{F}_{\text{OCS}}, \quad 1 \leq m \leq M.$$

Here, $\mathcal{Q}$ ranges over valid segmentations with potentially different numbers of segments. The communication and reconfiguration components of the objective are defined in Equation 5.

The formulation jointly captures two decisions. The segment boundaries determine when the optical fabric is reconfigured, while $\mathbf{X}^{(m)}$ determines how the available directional connection units are assigned during segment m . The resulting configuration may be symmetric or asymmetric, depending on the ordered demands of the communication nodes contained in that segment.

The output is a directional connectivity schedule

$$\mathcal{S} = \left\{ \left(\mathcal{I}_m, \mathbf{X}^{(m)} \right) \right\}_{m=1}^M, \quad (7)$$

where each entry specifies a contiguous communication-node segment and the integer OCS configuration used throughout that segment.

The next section presents DRAC. It profiles the ordered demand of every communication node, derives the corresponding continuous directional targets, partitions the target sequence into configuration segments, realizes the selected targets as feasible integer OCS configurations, and compacts their schedule-wide physical resource footprint.

IV. DRAC DESIGN

DRAC is an offline planning framework that converts the deterministic communication structure of an LLM training job into the directional connectivity schedule defined in Equation 7. DRAC operates below the collective communication layer. It determines when the optical fabric should be reconfigured, how many connection units should be assigned to each ordered GPU–NIC endpoint pair, and which physical optical resources can be released from the current job.

DRAC operates below the collective communication layer. It does not modify the training program, collective semantics, rank placement, or GPU–NIC association. Instead, it extracts the ordered point-to-point demand generated by the existing communication schedule and configures the OCS-facing optical channels accordingly.

A. Design Overview

Directly solving the provisioning problem in Equation 6 is difficult because the segment boundaries and the integer directional configurations are coupled. Determining whether a

new segment boundary is worthwhile depends on the configuration available to the resulting segment, while finding that configuration requires solving a discrete allocation problem over limited endpoint Tx and Rx channels.

DRAC decomposes this joint problem into four stages, as shown in FigureXX:

First, the *Ordered Demand Profiler* extracts the communication subgraph from the static training computation graph. Each communication node is expanded into its directed point-to-point transfers and represented by one ordered demand matrix.

Second, the *Directional Target Generator* computes an optimal continuous directional target for every communication node. These node-level targets form a target sequence. DRAC then uses dynamic programming to group consecutive communication nodes whose targets can efficiently share one configuration.

Third, the *OCS Realization Engine* converts each selected continuous target into a sparse and feasible integer OCS configuration. Rather than consuming every available channel, it uses only enough connection units to keep the realized segment cost within a configurable tolerance of the continuous target.

Finally, the *Schedule-Wide Resource Compaction Planner* maps all segment configurations onto the smallest stable physical channel pool that can support the complete schedule. The remaining channels are exposed to the cluster-level resource manager.

All optimization is performed offline when the model, parallelism configuration, rank placement, collective implementation, or optical resource inventory changes. The resulting schedule is reused across recurring training iterations.

B. Ordered Demand Profiler

The input to the profiler is the computation graph defined in section III-A.

For a point-to-point communication operation (such as PP), the communication schedule directly identifies the source and destination ranks. DRAC maps the communicating ranks to their corresponding GPU-NIC endpoints and records the tensor volume in the appropriate ordered matrix entry. For a collective operation, the logical collective API alone does not expose the ordered point-to-point transfers used by its implementation. DRAC therefore expands the collective according to its deterministic communication schedule.

For example, a ring-based DP synchronization specifies the predecessor and successor of every participating rank, as well as the tensor chunks transmitted during each ring step. A chunk sent from rank r_i to rank r_j contributes only to the ordered direction from the endpoint hosting r_i to the endpoint hosting r_j . The reverse direction remains a separate demand entry.

The logical tensor volume is augmented using an offline transport-calibration model:

$$T_{uv}^{(k)} = T_{uv,\text{payload}}^{(k)} + T_{uv,\text{ctrl}}^{(k)}.$$

The second term accounts for packet headers, acknowledgements, credit messages, and other control traffic, including lightweight reverse-direction traffic. Calibration is performed once for a given NIC, transport configuration, message-size range, and collective implementation.

After profiling, traffic whose source and destination belong to the same server is removed according to the system model in Section III. The profiler does not classify an endpoint pair as symmetric or asymmetric and does not apply a skew threshold. The output is one ordered traffic matrix for every communication node:

$$\mathbf{T}^{(1)}, \mathbf{T}^{(2)}, \dots, \mathbf{T}^{(K)}.$$

The communication-node expansion and transport-calibration procedure are described in Appendix IX-B.

C. Directional Target Generator

The Directional Target Generator first computes an ideal continuous allocation independently for every communication node. It then treats the resulting allocation matrices as a target sequence and determines where the sequence should be partitioned.

1) *Per-Node Continuous Target*: For communication node p_k , DRAC relaxes the integer connection matrix $\mathbf{X}$ into a continuous matrix $\mathbf{Y}^{(k)}$. The continuous feasible set $\mathcal{F}_{\text{cont}}$ is obtained by replacing the integrality requirement in $\mathcal{F}_{\text{OCS}}$ with $Y_{uv}^{(k)} \geq 0$, while retaining the Tx-, Rx-, and inter-server constraints defined in Section III.

Let $\tilde{L}_k(\mathbf{Y})$ denote the completion time defined by Equation 4 after replacing the integer configuration $\mathbf{X}$ with the continuous allocation $\mathbf{Y}$. The node-level target is

$$\mathbf{Y}^{*(k)} = \arg \min_{\mathbf{Y} \in \mathcal{F}_{\text{cont}}} \tilde{L}_k(\mathbf{Y}).$$

For the common case in which $B_{uv}^{\text{fix}} = 0$, the optimal completion time is determined by the most heavily loaded Tx or Rx endpoint:

$$\Theta_k^{\text{tx}} = \max_u \frac{\sum_v T_{uv}^{(k)}}{cn_u^{\text{tx}}}, \Theta_k^{\text{rx}} = \max_v \frac{\sum_u T_{uv}^{(k)}}{cn_v^{\text{rx}}}.$$

The optimal node completion time and one minimum-resource optimal target are

$$\Theta_k^* = \max \{\Theta_k^{\text{tx}}, \Theta_k^{\text{rx}}\}, \quad Y_{uv}^{*(k)} = \frac{T_{uv}^{(k)}}{c\Theta_k^*}. \quad (8)$$

Equation 8 directly relates the allocated directional capacity to the ordered traffic demand. Directions carrying more traffic receive proportionally more capacity, while the scaling factor is determined by the bottleneck endpoint budget.

The target is minimum-resource in the following sense: it allocates only the capacity required to achieve the optimal completion time and does not consume slack resources at non-bottleneck endpoints. Therefore, the continuous target itself does not artificially exhaust all available optical channels. The optimality proof and the extension to nonzero B_{uv}^{fix} are provided in Appendix IX-C.

2) *Target-Sequence Segmentation*: After solving every communication node, DRAC obtains the continuous target sequence

$$\mathbf{Y}^{*(1)}, \mathbf{Y}^{*(2)}, \dots, \mathbf{Y}^{*(K)}.$$

To evaluate whether two or more consecutive communication nodes can share one target, DRAC first constructs a target service-cost matrix:

$$D_{kh} = \tilde{L}_k \left(\mathbf{Y}^{*(h)} \right).$$

Here, D_{kh} is the completion time of communication node p_k when it is served using the target originally optimized for node p_h . For a candidate segment $[s, e]$, DRAC selects the best representative target among the node-level targets contained in the segment:

$$h^*[s, e] = \arg \min_{s \leq h \leq e} \sum_{k=s}^e D_{kh}.$$

The corresponding candidate-segment cost is

$$\tilde{C}[s, e] = \sum_{k=s}^e D_{k, h^*[s, e]}. \quad (9)$$

This construction uses the node-level continuous solutions as the direct input to segmentation. It does not re-solve a new continuous allocation for every possible segment.

DRAC then uses dynamic programming. Let $\text{OPT}[e]$ denote the minimum logical cost for the prefix $p_1, \dots, p_e$. The recurrence is

$$\text{OPT}[e] = \min_{0 \leq q < e} \left\{ \text{OPT}[q] + \tilde{C}[q+1, e] + \mathbb{I}[q > 0] \delta \right\}, \quad (10)$$

with $\text{OPT}[0] = 0$. Backtracking produces the final boundaries $0 = q_0 < q_1 < \dots < q_M = K$. For segment $\mathcal{I}_m$, the selected representative index and target are

$$h_m = h^*[q_{m-1} + 1, q_m], \quad \mathbf{Y}^{*(m)} = \mathbf{Y}^{*(h_m)}.$$

The output of the Directional Target Generator is

$$\mathcal{S}_{\text{logical}} = \left\{ \left(\mathcal{I}_m, \mathbf{Y}^{*(m)} \right) \right\}_{m=1}^M.$$

For the segment-cost definition in Equation 9, the dynamic program returns the globally minimum-cost contiguous partition and representative-target selection. The complete procedure and proof follow Algorithm 1 in Appendix IX-C.

This optimality is scoped to the continuous target dictionary $\{\mathbf{Y}^{*(k)}\}$. The later integer realization remains a quantized approximation of the selected targets.

D. OCS Realization Engine

The OCS Realization Engine converts each selected continuous target $\mathbf{Y}^{*(m)}$ into an integer configuration $\mathbf{X}^{(m)}$.

A realization policy that repeatedly fills every positive fractional residual may consume all remaining Tx and Rx channels, even after the realized communication performance is already sufficiently close to the continuous target. This

would unnecessarily reduce the resources available to other jobs.

DRAC instead formulates realization as a performance-constrained resource-minimization problem:

$$\min_{\mathbf{X} \in \mathcal{F}_{\text{OCS}}} \|\mathbf{X}\|_1 \quad \text{s.t.} \quad C_m(\mathbf{X}) \leq (1 + \epsilon) \tilde{C}_m, \quad (11)$$

where $\tilde{C}_m$ is the logical cost of the selected continuous target and ϵ is the allowed realization slowdown.

A smaller ϵ more closely preserves the continuous target, while a larger ϵ allows DRAC to use fewer optical connection units. The special case $\epsilon = 0$ requests the best attainable match to the continuous target.

The exact integer problem remains combinatorial. DRAC uses a target-bounded greedy procedure. It begins from a conservative floor allocation, prioritizes positive-demand directions that otherwise have zero realized capacity, and then adds the connection with the largest marginal segment-cost reduction. Unlike residual-filling schemes that run until no residual remains, DRAC stops as soon as Equation 11 is satisfied. A reverse-pruning pass then removes any connection unit whose removal preserves the performance constraint. Consequently, unused channels are retained intentionally rather than merely appearing when the allocation happens not to consume them.

If the available integer resources cannot meet the requested tolerance, the engine returns the best feasible configuration and records the unsatisfied segment as resource-constrained. The exact marginal-gain expression and the complete procedure are given by Algorithm 2 in Appendix IX-D.

The two ordered directions remain independent throughout realization. Hence, $Y_{uv}^{*(m)} > Y_{vu}^{*(m)}$ may naturally produce $X_{uv}^{(m)} > X_{vu}^{(m)}$. This asymmetry is realized by assigning different numbers of fixed-rate connection units. DRAC neither changes the rate of an optical channel nor converts an Rx channel into a Tx channel. The realization engine outputs

$$\mathcal{S}_{\text{OCS}} = \left\{ \left(\mathcal{I}_m, \mathbf{X}^{(m)} \right) \right\}_{m=1}^M.$$

E. Schedule-Wide Resource Compaction and Exposure

Per-segment idle channels are not necessarily releasable, because another segment may require them later in the same training iteration. DRAC therefore considers the complete OCS schedule and constructs the smallest stable channel pool that can instantiate every selected configuration.

Let $\bar{n}_u^{\text{tx}}$ and $\bar{n}_u^{\text{rx}}$ denote the numbers of physical Tx and Rx channels reserved for endpoint u throughout the job. The planner solves

$$\min \sum_u (\bar{n}_u^{\text{tx}} + \bar{n}_u^{\text{rx}}), \quad (12)$$

subject to every segment configuration being embeddable in the reserved pools. The optimal reservation has the closed form

$$\bar{n}_u^{\text{tx}} = \max_m \sum_{v \neq u} X_{uv}^{(m)}, \quad (13)$$

$$\bar{n}_u^{\text{rx}} = \max_m \sum_{v \neq u} X_{vu}^{(m)}. \quad (14)$$

These values are not merely utilization statistics. They are the minimum stable physical inventories that can support all segment configurations without reallocating channel ownership between segment boundaries.

The maximal numbers of channels that can be safely exposed from the current job are therefore

$$\begin{aligned} f_u^{\text{tx}} &= n_u^{\text{tx}} - \bar{n}_u^{\text{tx}}, \\ f_u^{\text{rx}} &= n_u^{\text{rx}} - \bar{n}_u^{\text{rx}}. \end{aligned}$$

Under a nonblocking OCS, a fixed pool of these peak sizes is also sufficient: each segment can rematch the reserved Tx and Rx channels without changing channel ownership. Therefore, Equations 13 and 14 expose the maximum stable-unused directional resources compatible with the complete schedule.

The planner also collects resource-constrained segments reported by the realization engine. For each such segment, it records the ordered directions with the largest remaining marginal gains. This directional request map informs future job placement, admission control, or optical-resource expansion decisions.

The optimality proof, physical channel-pool construction, and segment-level binding procedure are given by Algorithm 3 in Appendix IX-E.

V. EVALUATION

In this section, we evaluate whether DRAC can translate directional training demand into deployable asymmetric OCS connectivity and whether such connectivity improves communication efficiency under port-limited AI fabrics. Our evaluation is organized around six questions. First, do training workloads contain directional skew at the ordered node-pair level? Second, can DRAC exploit such skew to reduce communication time? Third, does DRAC better match provisioned capacity to directional demand? Fourth, how much symmetric capacity waste can DRAC reduce? Fifth, how close does the realized OCS allocation remain to the logical target under limited port budgets? Finally, does DRAC avoid harming nearly symmetric PP traffic?

A. Methodology

Workloads. We evaluate DRAC using LLaMA-3-based transformer training workloads. The workloads include tensor parallelism (TP), data parallelism (DP), mixed TP+DP communication, and pipeline parallelism (PP) under a 1F1B schedule. For each workload, we derive the communication operations from the training graph and construct the ordered node-pair demand matrix according to the corresponding parallelism strategy. The resulting workloads capture the recurring communication patterns of LLaMA-3 training while allowing us to isolate how different parallel domains affect directional demand.

Network model. We model an OCS-enabled AI fabric in which each abstract node has a limited number of outbound and inbound OCS connection units. Each OCS connection unit provides bandwidth c . DRAC computes a segment-level

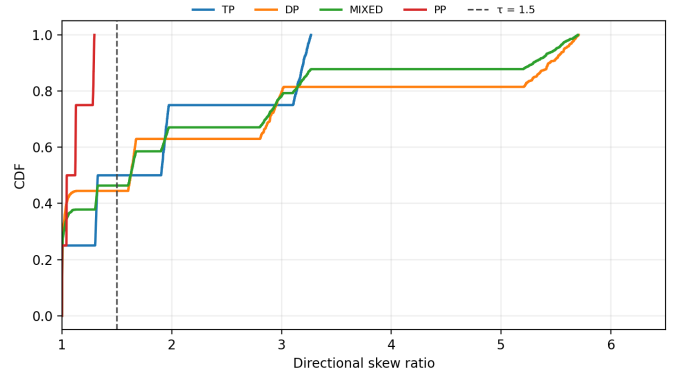


Fig. 6. Directional skew distribution across workloads. Only off-diagonal ordered node pairs are included. The dashed line marks the skew threshold $\tau = 1.5$.

logical target B^* and realizes it as an integer OCS connection matrix X^* . When communication is executed across multiple segments, OCS reconfiguration overhead is charged at segment boundaries. Unless otherwise specified, all schemes use the same total OCS resource budget.

Schemes. We compare DRAC with two symmetric baselines.

Static-Sym is a fixed symmetric network that provisions the same capacity for both directions of each connected node pair. It represents conventional static AI fabrics without direction-aware reconfiguration.

Sym-OCS is an ACTINA-style symmetric OCS baseline. It uses the same OCS resource budget and the same segment-level planning model as ACTINA, it allocates capacity symmetrically between opposite directions of each node pair. This baseline isolates the benefit of direction-aware asymmetric provisioning from the benefit of OCS reconfigurability itself.

DRAC uses the same OCS resource budget, but treats the two ordered directions of a node pair as independent provisioning objects. It computes a directional logical target and realizes the target through integer OCS connection units.

B. Directional Skew Characterization

We first examine whether training traffic contains sufficient directional skew to justify asymmetric provisioning. For each unordered node pair, we compute the directional skew ratio ρ_{ij} using only off-diagonal ordered traffic entries. A value close to one indicates balanced bidirectional demand, while a larger value indicates stronger directional imbalance. DRAC selects high-demand directions using the thresholds τ and η defined in Section ??.

Figure 6 shows the skew distribution across TP, DP, MIXED, and PP workloads. TP, DP, and MIXED all contain a non-negligible fraction of directions whose skew ratio exceeds $\tau = 1.5$, indicating that the two directions of many node pairs do not carry equal demand. In contrast, PP traffic is concentrated near $\rho = 1$, which confirms that PP is much closer to bidirectionally balanced under the evaluated 1F1B

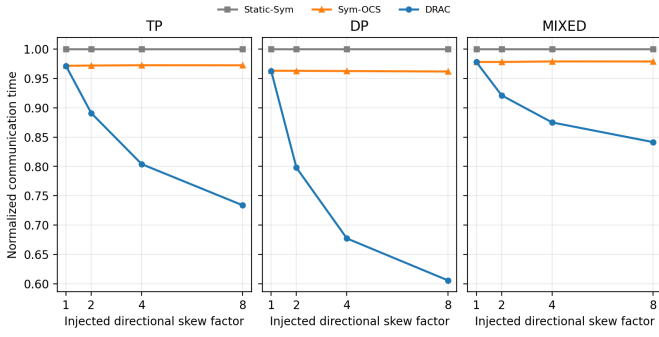


Fig. 7. Normalized communication time under injected directional skew. Results are normalized to Static-Sym.

schedule. This result supports the central design choice of DRAC: asymmetric provisioning should be applied opportunistically to skewed communication domains rather than forced onto all traffic patterns.

C. Communication Time under Directional Skew

We next evaluate whether DRAC can exploit directional skew to reduce communication time. To isolate the effect of skew, we use a controlled sensitivity experiment that injects different directional skew factors into TP, DP, and MIXED workloads. For each skew factor, we compare Static-Sym, Sym-OCS, and DRAC under the same OCS resource budget.

Figure 7 shows that DRAC behaves similarly to Sym-OCS when the injected skew factor is close to one. This is the desired behavior: when the demand is nearly symmetric, DRAC has little opportunity to reassign capacity across directions. As directional skew increases, however, DRAC achieves increasingly lower communication time than Sym-OCS. The improvement is especially clear for DP, where the high-demand directions dominate communication time and symmetric reverse capacity becomes less useful. Static-Sym and Sym-OCS remain relatively flat because they cannot reassign capacity independently between opposite ordered directions. This demonstrates that DRAC is opportunistic: it extracts benefit when directional skew exists, but falls back to near-symmetric behavior when the demand is balanced.

D. Directional Bandwidth Matching

We then evaluate whether DRAC improves the alignment between ordered traffic demand and realized OCS capacity. For each selected high-demand direction, we compute the demand share and realized capacity share. The per-entry matching error is the absolute difference between the two shares, weighted by traffic demand. Lower error means that the realized OCS allocation better preserves the directional structure of the training workload.

Figure 8 reports the CDF of weighted directional share error. DRAC shifts the error distribution to the left in TP and MIXED, showing that it better matches provisioned capacity to the dominant ordered directions. In DP, DRAC remains

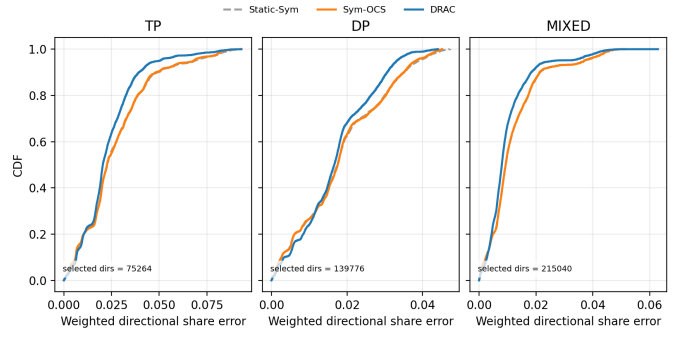


Fig. 8. CDF of weighted directional share error over selected high-demand directions. PP is omitted because no high-demand directions are selected under $\tau = 1.5$.

comparable to or slightly better than Sym-OCS, indicating that the discrete OCS realization preserves most of the directional target even when the matching problem is constrained by port budgets. Since the CDF is computed only over selected high-demand directions, the result is not diluted by zero-volume or low-volume entries. This directly evaluates the directions that DRAC is designed to improve.

To provide a more intuitive view, we also visualize a representative DP segment. The segment is selected as the DP segment with the largest aggregate matching-error reduction by DRAC. The diagonal is masked because self traffic does not consume OCS-facing connectivity.

Figure 9 shows that the demand matrix contains clear directional bands, meaning that traffic is concentrated on a small set of ordered directions rather than evenly spread across all bidirectional pairs. Sym-OCS can adapt which node pairs receive OCS capacity, but its symmetric allocation limits how well it can follow the ordered demand. DRAC assigns capacity more directly to the dominant directions, and the error-reduction heatmap shows where this improves the match between realized capacity share and demand share. This visual comparison confirms that DRAC changes not only the amount of OCS capacity assigned to a communication phase, but also its directional placement.

E. Symmetric Capacity Waste

We next quantify the capacity wasted by symmetric provisioning. For each ordered direction, useful capacity is the portion of provisioned bandwidth needed to complete its traffic within the communication window, while excess provisioned capacity is counted as waste. The waste ratio is the total wasted capacity divided by the total provisioned directional capacity. Lower waste ratio indicates that more of the provisioned capacity is placed on useful directions.

Figure 10 shows that DRAC consistently reduces capacity waste compared with Static-Sym and Sym-OCS. Static-Sym wastes capacity because it cannot adapt to the training demand. Sym-OCS improves placement at the node-pair level, but still reserves capacity symmetrically within each selected

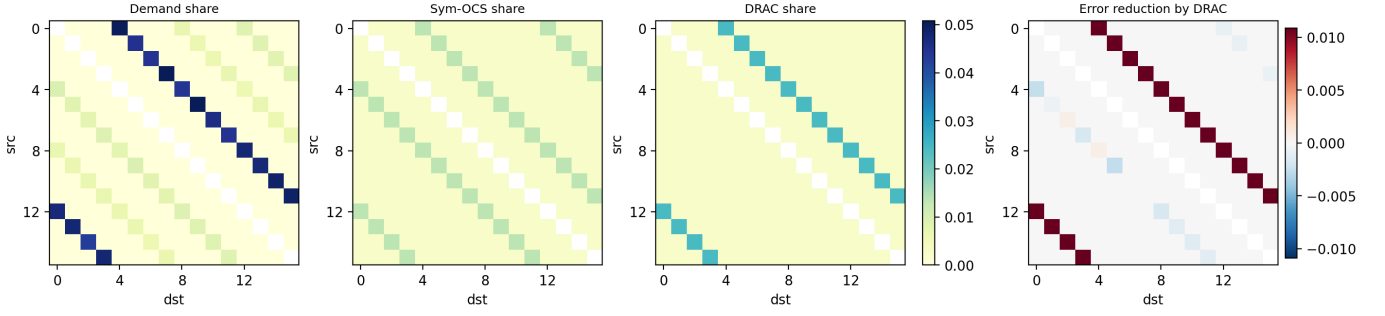


Fig. 9. Representative DP segment: demand share, Sym-OCS share, DRAC share, and per-entry error reduction by DRAC. Positive values in the last heatmap indicate lower error under DRAC.

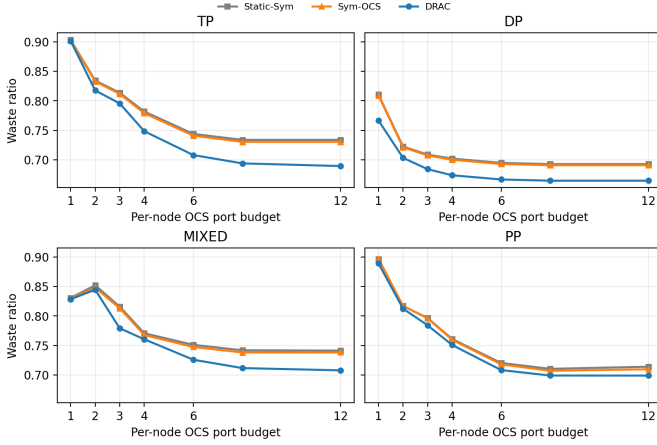


Fig. 10. Capacity waste ratio under different per-node OCS port budgets.

pair. DRAC reduces waste by shifting OCS connection units from low-demand reverse directions to high-demand forward directions. The benefit persists across different port budgets, showing that direction-aware provisioning remains useful even as more OCS resources become available. For PP, the gain is smaller, which is consistent with the weaker directional skew observed in Figure 6.

F. Residual Target Gap under OCS Discretization

The previous results show that DRAC improves capacity placement, but the logical target B^* is continuous while OCS resources are discrete. We therefore evaluate how much of the logical target remains unrealized after the realization engine maps B^* to integer OCS connection units. For each selected high-demand direction, we compute the residual gap $G_{ij} = [B_{ij}^* - \hat{B}_{ij}]^+$ and normalize the aggregate gap by the total logical target over selected directions.

Figure 11 shows that the residual target gap decreases as the per-node OCS port budget increases. This confirms that the remaining mismatch is largely caused by finite OCS resources and integer connection granularity. TP and DP both benefit from additional ports, while MIXED remains more

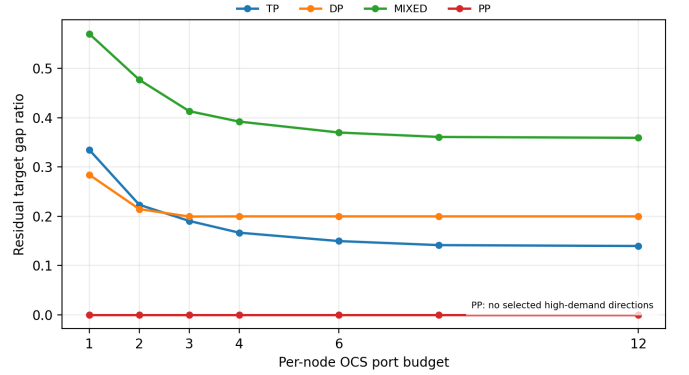


Fig. 11. Residual target gap ratio over selected high-demand directions.

difficult because its demand combines multiple communication structures and therefore spreads the logical target across more directions. PP has no selected high-demand directions under the skew threshold, so its residual gap is reported as zero. This result shows that the realization engine provides a useful signal to the resource manager: persistent residual gaps identify where additional OCS resources would be most valuable in future planning epochs.

G. Iso-Performance OCS Port Saving

We further evaluate DRAC from a resource-efficiency perspective. Instead of asking how many physical resources are unused under a fixed reservation, we ask how many OCS ports are required to reach the same communication performance. Specifically, we use the communication time achieved by Sym-OCS with six ports per node as the target, and then find the minimum number of ports required by DRAC to reach the same or lower communication time.

Figure 12 shows that DRAC reaches the Sym-OCS six-port performance target with fewer ports for skewed workloads. DRAC requires only four ports for TP, two ports for DP, and four ports for MIXED. This indicates that symmetric OCS provisioning uses part of its port budget on low-value reverse directions, whereas DRAC can concentrate fewer connection

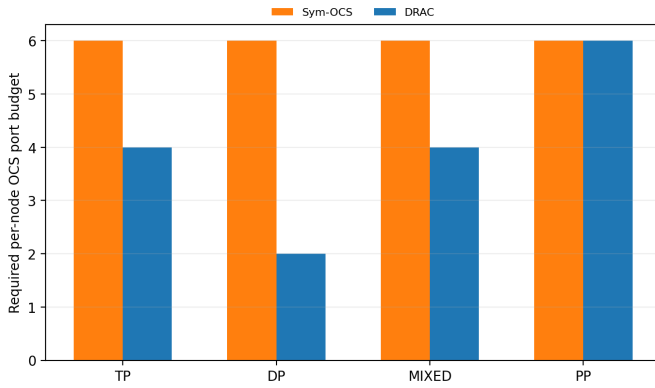


Fig. 12. Required per-node OCS ports to reach the communication time achieved by Sym-OCS with six ports per node.

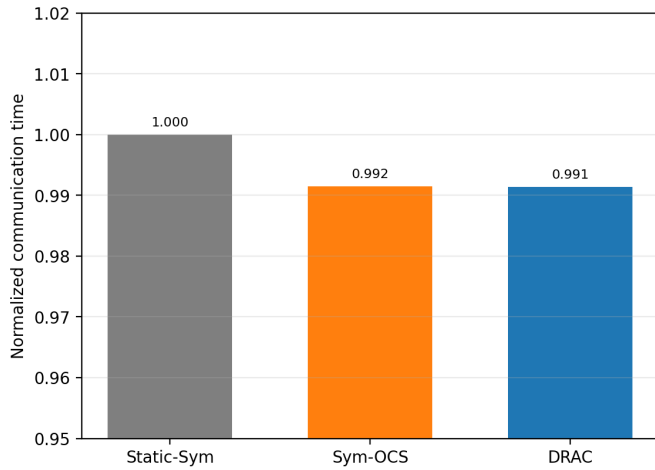


Fig. 13. PP no-harm case study under near-symmetric PP demand.

units on the directions that determine communication time. In contrast, PP requires the same number of ports as Sym-OCS. This is expected because PP traffic is close to bidirectionally balanced and therefore provides little opportunity for direction-aware port reduction. The result demonstrates that DRAC’s resource savings come from directional skew rather than from reducing capacity indiscriminately.

H. Symmetric-Traffic Case Study: Pipeline Parallelism

Finally, we evaluate DRAC under PP traffic, where the forward and backward stages of the 1F1B schedule make the demand much closer to bidirectionally balanced. This experiment tests whether DRAC harms a workload that is not expected to benefit strongly from asymmetric provisioning.

Figure 13 shows that DRAC remains essentially identical to Sym-OCS under PP. The difference between DRAC and Sym-OCS is within a very small margin, indicating that DRAC does not force unnecessary asymmetry when the workload itself is nearly symmetric. Combined with Figure 6, this confirms that

DRAC is selective: it exploits directional imbalance in TP, DP, and MIXED, while preserving near-symmetric behavior for PP.

I. Summary

The evaluation demonstrates five main findings. First, TP, DP, and MIXED workloads contain significant ordered-direction skew, whereas PP is much closer to bidirectionally balanced. Second, DRAC’s benefit grows with directional skew and remains close to symmetric OCS when skew is weak. Third, DRAC improves directional bandwidth matching on selected high-demand directions and visually aligns realized capacity with dominant demand directions in representative DP segments. Fourth, DRAC reduces symmetric capacity waste across OCS port budgets. Fifth, DRAC can achieve the same communication performance as Sym-OCS with fewer OCS ports for skewed workloads, while requiring no artificial port saving under PP. Overall, these results show that DRAC realizes direction-aware asymmetric connectivity without changing collective communication semantics and without harming near-symmetric traffic.

VI. RELATED WORKS

Existing research on networks for large-scale AI training has mainly followed three directions: collective communication and logical topology optimization, workload-aware OCS reconfiguration, and OCS fabric architecture and physical deployment. Table X summarizes representative approaches from three perspectives. The logical layer concerns the generation or optimization of workload-level communication requirements and logical communication structures. The realization layer concerns the translation of logical requirements into executable network configurations. The physical layer concerns optical-network capabilities, deployment constraints, and physical-resource management. This classification provides a common view of prior work without requiring approaches in the same layer to provide identical functionality.

A. Collective Communication and Topology Optimization

Communication optimization for large-scale AI training has focused on collective execution and workload-aware topology design. AdapCC [23] uses GPU placement, link conditions, and runtime profiling to adapt collective communication graphs, data-chunking strategies, and execution policies. It improves collective performance over a given network. TopoOpt [21] further co-optimizes the training parallelization strategy, communication traffic, network topology, and routing plan. It therefore constructs a network structure that better matches the communication pattern of a specific training workload. These approaches mainly operate at the logical layer, while TopoOpt also partially translates workload requirements into a direct-connect topology and its routing plan.

The main optimization targets of these approaches are collective execution and the overall network topology. They determine how communication should be organized or which topology should serve a training job. However, they generally

do not address how directional communication requirements should be realized with discrete OCS connectivity resources. DRAC instead focuses on connectivity provisioning under a given training communication pattern, allowing the configured connectivity to reflect directional differences in ordered node-pair demand.

B. Workload-Aware OCS Reconfiguration for AI Training

The repetitive and predictable structure of AI training communication enables network configurations to be planned according to workload requirements. ACTINA [22] exploits the deterministic sequence of communication operations in a training computation graph. It dynamically allocates bandwidth among TP, DP, and PP communication domains and uses pre-planned scheduling to balance communication benefits against OCS reconfiguration overhead. MixNet [10] targets dynamic all-to-all communication in distributed MoE training and adjusts regional OCS connectivity according to traffic generated by expert parallelism. Opus [4] exploits communication changes across training phases and switches photonic-rail topologies at phase boundaries. These systems perform workload-aware optical reconfiguration at the communication-domain, regional, and training-phase granularities, respectively.

DRAC also exploits predictable training communication, but differs in its provisioning object and output granularity. ACTINA reallocates bandwidth among communication domains, MixNet adjusts optical connectivity within selected MoE regions, and Opus switches rail-level topologies across training phases. These approaches mainly determine where and when network resources should be adjusted. DRAC instead treats the two ordered directions of a node pair as separate provisioning objects and generates direction-aware OCS connectivity at the port level. It therefore extends workload-aware reconfiguration from domain-, region-, or phase-level adaptation to realization at the granularity of ordered directions.

DRAC builds on ACTINA’s graph-driven planning principle by using the known communication sequence to balance provisioning benefits against reconfiguration overhead. The key distinction is the allocation granularity. ACTINA allocates bandwidth among communication domains, whereas DRAC allocates connectivity among ordered node-pair directions and realizes the resulting directional requirements as discrete OCS configurations.

C. OCS-Enabled Datacenter and AI Fabrics

Another line of research focuses on the architecture, scalability, and deployment of OCS-enabled networks. Apollo [19] demonstrates production-scale OCS deployment in datacenter networks. It addresses MEMS-based OCS design, WDM transceivers, circulators, insertion loss, switching latency, and bidirectional optical links. InfiniteHBD [18] targets large-scale LLM clusters and constructs a datacenter-scale high-bandwidth domain using OCS transceivers and optical interconnects. These systems explicitly address optical devices,

port organization, physical topology, and deployment feasibility.

The primary research object of Apollo and InfiniteHBD is the OCS infrastructure. Apollo studies how OCS can be reliably deployed at datacenter scale, whereas InfiniteHBD studies how to construct a scalable high-bandwidth optical interconnect for LLM clusters. Their main goal is to provide the physical and architectural capabilities required for large-scale reconfigurable connectivity.

DRAC assumes that such reconfigurable connectivity is available and studies how it should be configured for a particular training workload. Apollo and InfiniteHBD define the physical connectivity capabilities that an OCS fabric can provide, whereas DRAC determines how those capabilities should be assigned across communication directions. Thus, the former primarily address the construction and deployment of the optical fabric, while DRAC addresses the configuration gap between workload demand and OCS connectivity.

Overall, prior work covers different portions of directional connectivity provisioning. AdapCC and TopoOpt primarily optimize logical communication structures and network topology. ACTINA, MixNet, and Opus use workload characteristics to drive OCS reconfiguration at the domain, region, or phase level. Apollo and InfiniteHBD provide the underlying OCS architecture and deployment capabilities. DRAC addresses the remaining realization gap by deriving direction-aware capacity requirements from ordered node-pair demand and translating them into deployable asymmetric OCS configurations. Its primary contribution is neither a new collective communication algorithm nor a new OCS hardware architecture, but a mechanism for realizing workload-level directional demand through executable OCS connectivity.

VII. CONCLUSION

This paper revisits the assumption that large-scale AI training fabrics should provision equal capacity in both directions of every node pair. We show that, in LLM training, collective communication can induce persistent directional skew at the ordered node-pair level, especially in TP and DP communication domains. This skew creates a mismatch between symmetric network provisioning and actual training demand under port-limited OCS fabrics.

We presented DRAC, a direction-aware reconfigurable asymmetric connectivity framework that maps ordered training demand to feasible OCS port matchings. DRAC derives a logical directional bandwidth target from recurring communication patterns, realizes this target as integer OCS connection units, and exposes the resulting asymmetric connectivity to the traffic steering layer. The design does not modify collective communication semantics or assume asymmetric physical link rates; instead, it realizes asymmetry by assigning different numbers of OCS connection units to opposite ordered directions.

Our evaluation on LLaMA-3-based workloads shows that DRAC improves communication efficiency when directional skew exists, reduces symmetric capacity waste, and reaches the same performance as symmetric OCS provisioning with

fewer ports for TP, DP, and mixed TP+DP workloads. At the same time, DRAC remains comparable to Sym-OCS under near-symmetric PP traffic, confirming that it does not force asymmetry where it is not beneficial. These results suggest that direction-aware asymmetric provisioning is a practical step toward AI-native network fabrics that match connectivity supply to the structured demand of large-scale LLM training.

VIII. ACKNOWLEDGEMENT

The authors would like to thank the anonymous reviewers for their constructive comments and valuable suggestions. We also thank the members of our research group for helpful discussions on OCS-enabled AI fabrics, large-scale training communication, and network resource provisioning.

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [3] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, “Palm: Scaling language modeling with pathways,” *Journal of machine learning research*, vol. 24, no. 240, pp. 1–113, 2023.
- [4] E. Ding, B. Lyu, B. Kataria, and R. Singh, “Photonic rails in ml datacenters with opus,” *arXiv preprint arXiv:2602.12521*, 2026.
- [5] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [6] X. Han, S. Zhao, Y. Lv, P. Cao, W. Jiang, S. Lin, and X. Wang, “Lumoscore: Highly scalable llm clusters with optical interconnect,” *CoRR*, 2024.
- [7] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefler, “Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms,” in *2025 IEEE Symposium on High-Performance Interconnects (HOTI)*. IEEE, 2025, pp. 48–59.
- [8] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu *et al.*, “Gpipe: Efficient training of giant neural networks using pipeline parallelism,” *Advances in neural information processing systems*, vol. 32, 2019.
- [9] J. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong *et al.*, “{MegaScale}: Scaling large language model training to more than 10,000 {GPUs},” in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 745–760.
- [10] X. Liao, Y. Sun, H. Tian, X. Wan, Y. Jin, Z. Wang, Z. Ren, X. Huang, W. Li, K. F. Tse *et al.*, “Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training,” in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 554–574.
- [11] T. Liu, C. Hei, F. Li, C. Gao, J. Cao, T. Wang, E. Zhai, and X. Wang, “Resccl: Resource-efficient scheduling for collective communication,” in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 55–70.
- [12] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, “Pipedream: Generalized pipeline parallelism for dnn training,” in *Proceedings of the 27th ACM symposium on operating systems principles*, 2019, pp. 1–15.
- [13] D. Narayanan, M. Shoyebi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro *et al.*, “Efficient large-scale language model training on gpu clusters using megatron-lm,” in *Proceedings of the international conference for high performance computing, networking, storage and analysis*, 2021, pp. 1–15.
- [14] NVIDIA Corporation, “NVIDIA NVSwitch: The World’s Highest-Bandwidth On-Node Switch,” NVIDIA Corporation, Technical Overview, 2018. [Online]. Available: <https://images.nvidia.com/content/pdf/nvswitch-technical-overview.pdf>
- [15] NVIDIA Corporation, “Megatron-lm: Training large transformer language models at scale,” <https://github.com/NVIDIA/Megatron-LM>, 2019, accessed: 2026-03-05.
- [16] NVIDIA Corporation, “Nvidia collective communications library (nccl) user guide: Overview,” <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/overview.html>, 2024, accessed: 2026-03-05.
- [17] L. Poutievski, O. Mashayekhi, J. Ong, A. Singh, M. Tariq, R. Wang, J. Zhang, V. Beaugregard, P. Conner, S. Gribble *et al.*, “Jupiter evolving: transforming google’s datacenter network via optical circuit switches and software-defined networking,” in *Proceedings of the ACM SIGCOMM 2022 Conference*, 2022, pp. 66–85.
- [18] C. Shou, G. Liu, H. Nie, H. Meng, Y. Zhou, Y. Jiang, W. Lv, Y. Xu, Y. Lu, Z. Chen *et al.*, “Infinitehbd: building datacenter-scale high-bandwidth domain for llm with optical circuit switching transceivers,” in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 1–23.
- [19] R. Urata, H. Liu, K. Yasumura, E. Mao, J. Berger, X. Zhou, Y. C. Lam, R. Bannan, D. Hutchinson, D. Nelson *et al.*, “Mission apollo: landing optical circuit switching at datacenter scale,” *arXiv preprint arXiv:2208.10041*, 2022.
- [20] W. Wang, M. Ghobadi, K. Shakeri, Y. Zhang, and N. Hasani, “Rail-only: A low-cost high-performance network for training llms with trillion parameters,” in *2024 IEEE Symposium on High-Performance Interconnects (HOTI)*. IEEE, 2024, pp. 1–10.
- [21] W. Wang, M. Khazraee, Z. Zhong, M. Ghobadi, Z. Jia, D. Mudigere, Y. Zhang, and A. Kewitsch, “{TopoOpt}: Co-optimizing network topology and parallelization strategy for distributed training jobs,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 739–767.
- [22] Z. Wu, B. Klenk, L. Dennison, and K. Bergman, “Actina: Adapting circuit-switching techniques for ai networking architectures,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1211–1222.
- [23] X. Zhao, Z. Zhang, and C. Wu, “Adapcc: Making collective communication in distributed machine learning adaptive,” in *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2024, pp. 25–35.

IX. APPENDIX

A. Sensitivity to Directional Traffic Skew

The representative DP example in Figure 5 exhibits substantial directional skew. We further evaluate how the cost of symmetric provisioning changes across a wider range of ordered-direction demand ratios. This experiment keeps the total traffic volume fixed and varies the fraction s carried by the main direction, with $s \in [0.5, 0.999]$. The opposite direction carries the remaining fraction $1 - s$.

Both strategies use the same budget of eight equal-rate unidirectional connection units. Symmetric provisioning assigns four units to each direction. Direction-aware provisioning enumerates all feasible integer allocations while retaining at least one unit for each active direction. For an allocation of x units to the main direction and $8 - x$ units to the opposite direction, the communication completion time is modeled as

$$\Theta(s, x) = \max \left(\frac{sT}{xC}, \frac{(1-s)T}{(8-x)C} \right), \quad (15)$$

where T is the fixed aggregate traffic volume and C is the bandwidth of one connection unit. The direction-aware allocation minimizes Equation 15 over the feasible integer values of x . The experiment excludes OCS reconfiguration latency and competition with other endpoint pairs so that it isolates the effect of the symmetry constraint.

Figure 14 shows the completion-time reduction relative to the symmetric 4/4 allocation. When the demand is balanced

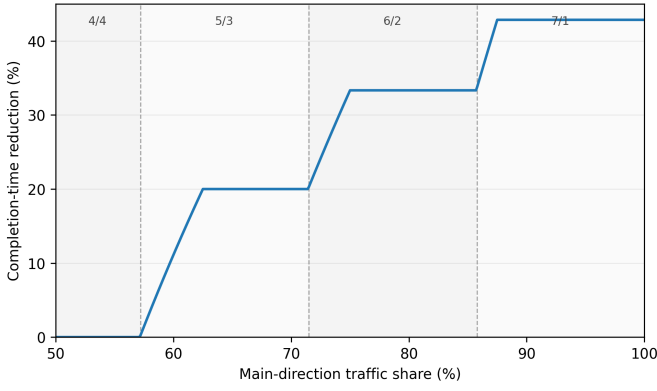


Fig. 14. Completion-time reduction over symmetric provisioning as the main-direction traffic share increases. The total traffic volume and the budget of eight equal-rate unidirectional connection units remain fixed.

at $s = 0.5$, direction-aware provisioning also selects 4/4, and the two strategies achieve the same completion time. As directional skew increases, the optimal allocation transitions successively through 5/3, 6/2, and 7/1. These transitions occur because optical connectivity is provisioned in indivisible units rather than as continuous capacity.

Within each allocation region, the benefit first increases while the opposite direction remains the completion-time bottleneck. Once the main direction becomes the bottleneck, the benefit reaches a plateau determined by the number of units assigned to that direction. The plateaus are 20.0% for 5/3, 33.3% for 6/2, and 42.9% for 7/1. For the final allocation, the 42.9% plateau begins when $s \geq 7/8$, because the main direction then determines completion time and receives seven units instead of the four available under symmetric provisioning.

The representative DP demand has a main-direction share of approximately

$$\frac{20.258}{20.258 + 0.144} \approx 99.3\%,$$

placing it within the 7/1 region. This explains the 42.9% reduction observed in Figure 5. The value is specific to the selected eight-unit budget and the requirement that at least one unit remain available to the active opposite direction; it is not a universal upper bound on the benefit of directional provisioning.

Overall, the sensitivity analysis confirms that direction-aware provisioning does not impose asymmetry when it is unnecessary. It reproduces the symmetric allocation under balanced demand and departs from it only when doing so reduces communication completion time.

B. Ordered-Demand Expansion and Transport Calibration

For communication node p_k , let $\mathcal{F}_k$ denote the directed transfers produced by its selected communication implementation. Define

$$\mathcal{F}_k(u, v) = \{f \in \mathcal{F}_k \mid \pi(\text{src}(f)) = u, \pi(\text{dst}(f)) = v\},$$

where $\pi(r)$ maps rank r to its GPU-NIC endpoint. The payload demand is then simply

$$T_{uv, \text{payload}}^{(k)} = \sum_{f \in \mathcal{F}_k(u, v)} \ell_f,$$

where ℓ_f is the payload size of transfer f .

DRAC stores two calibrated quantities for each communication environment and message-size bin:

$$h_{\text{fwd}}(\ell) \quad \text{and} \quad h_{\text{rev}}(\ell),$$

representing same-direction protocol overhead and reverse-direction control traffic, respectively. For every transfer $f : u \rightarrow v$, DRAC adds

$$\ell_f + h_{\text{fwd}}(\ell_f)$$

to $T_{uv}^{(k)}$ and

$$h_{\text{rev}}(\ell_f)$$

to $T_{vu}^{(k)}$.

The values are obtained from repeated isolated transfers by subtracting idle counter changes and taking the median directional byte increment. Message sizes not explicitly profiled use the nearest calibration bin.

C. Per-Node Target Solver and Segmentation

1) *Optimality of the Per-Node Target:* Assume first that $B_{uv}^{\text{fix}} = 0$. Any continuous allocation completing node p_k within time Θ must satisfy

$$cY_{uv} \geq \frac{T_{uv}^{(k)}}{\Theta}.$$

Summing this inequality over the destinations of endpoint u gives

$$\Theta \geq \frac{\sum_v T_{uv}^{(k)}}{cn_u^{\text{tx}}}.$$

Similarly, summing over the sources of endpoint v gives

$$\Theta \geq \frac{\sum_u T_{uv}^{(k)}}{cn_v^{\text{rx}}}.$$

Therefore, every feasible allocation satisfies

$$\Theta \geq \max \{ \Theta_k^{\text{tx}}, \Theta_k^{\text{rx}} \}.$$

The construction in Equation 8 satisfies every Tx and Rx constraint and completes all ordered transfers in exactly Θ_k^* or less. It therefore attains the lower bound and is globally optimal.

Moreover, every allocation achieving Θ_k^* must satisfy

$$Y_{uv} \geq \frac{T_{uv}^{(k)}}{c\Theta_k^*}.$$

Thus, Equation 8 is also the entry-wise minimum allocation among all optimal solutions.

When $B_{uv}^{\text{fix}} > 0$, the minimum augmentation required for a candidate completion time Θ is

$$Y_{uv}(\Theta) = \frac{[T_{uv}^{(k)}/\Theta - B_{uv}^{\text{fix}}]^+}{c}. \quad (16)$$

Algorithm 1 Target-Sequence Segmentation

Require: Node targets $\{\mathbf{Y}^{*(k)}\}_{k=1}^K$, demands $\{\mathbf{T}^{(k)}\}_{k=1}^K$, and delay δ

Ensure: Boundaries $\{q_m\}$ and representative indices $\{h_m\}$

```

1: for  $k = 1$  to  $K$  do
2:   for  $h = 1$  to  $K$  do
3:      $D_{kh} \leftarrow \tilde{L}_k(\mathbf{Y}^{*(h)})$ 
4:   end for
5: end for
6: for all  $1 \leq s \leq e \leq K$  do
7:    $h^*[s, e] \leftarrow \arg \min_{h \in [s, e]} \sum_{k=s}^e D_{kh}$ 
8:    $\tilde{C}[s, e] \leftarrow \sum_{k=s}^e D_{k, h^*[s, e]}$ 
9: end for
10:  $\text{OPT}[0] \leftarrow 0$ 
11: for  $e = 1$  to  $K$  do
12:   Evaluate Equation 10
13:   Record its minimizing predecessor
14: end for
15: Backtrack from  $K$  to recover  $\{q_m\}$ 
16: Set  $h_m \leftarrow h^*[q_{m-1} + 1, q_m]$ 
17: return  $\{q_m\}$  and  $\{h_m\}$ 

```

Feasibility is monotone in Θ . DRAC therefore finds the smallest feasible value by binary search over Θ and uses Equation 16 to construct the minimum-resource optimal target.

2) *Segmentation Procedure:* Algorithm 1 constructs the service-cost matrix, computes the representative target of each candidate segment, and applies the recurrence in Equation 10.

For an optimal solution ending at p_e , suppose its final segment starts after p_q . The prefix ending at p_q must itself be optimally segmented; otherwise replacing it with a lower-cost prefix would improve the full solution. This optimal-substructure property proves that Equation 10 returns the globally minimum segmentation under the candidate-target cost in Equation 9.

D. Target-Bounded Integer Realization

For an integer configuration $\mathbf{X}$, the marginal gain of adding one connection from u to v is

$$\Delta_{uv}^{(m)} = C_m(\mathbf{X}) - C_m(\mathbf{X} + \mathbf{E}_{uv}),$$

where $\mathbf{E}_{uv}$ contains one at entry (u, v) . Similarly, the cost increase caused by removing one assigned connection is

$$\Lambda_{uv}^{(m)} = C_m(\mathbf{X} - \mathbf{E}_{uv}) - C_m(\mathbf{X}).$$

Algorithm 2 approximates the resource-minimization problem in Equation 11.

The addition phase terminates immediately after the performance target is reached; it does not continue until all positive residuals or all physical channels are consumed. The reverse-pruning phase further removes connections that are not needed to preserve the requested performance tolerance.

Algorithm 2 Target-Bounded Sparse Realization

Require: Selected target $\mathbf{Y}^{*(m)}$, logical cost $\tilde{C}_m$, and tolerance ϵ

Ensure: Integer configuration $\mathbf{X}^{(m)}$

```

1:  $X_{uv}^{(m)} \leftarrow \lfloor Y_{uv}^{*(m)} \rfloor$ 
2: Add feasible units to positive-demand directions having zero realized capacity
3: while  $C_m(\mathbf{X}^{(m)}) > (1 + \epsilon)\tilde{C}_m$  and a feasible target-bounded addition exists do
4:   Compute  $\Delta_{uv}^{(m)}$  for feasible additions
5:   Add the connection with the largest gain
6: end while
7: if  $C_m(\mathbf{X}^{(m)}) \leq (1 + \epsilon)\tilde{C}_m$  then
8:   while a connection can be removed without violating the tolerance do
9:     Compute  $\Lambda_{uv}^{(m)}$ 
10:    Remove the connection with the smallest loss
11:   end while
12: else
13:   Mark segment  $m$  as resource-constrained
14: end if
15: return  $\mathbf{X}^{(m)}$ 

```

E. Schedule-Wide Physical Resource Compaction

For any endpoint u , every stable Tx pool must contain at least

$$\max_m \sum_v X_{uv}^{(m)}$$

channels, because one segment requires exactly that many simultaneous transmissions. The same lower bound applies to the Rx pool.

Conversely, under a nonblocking OCS, a fixed pool of these peak sizes is sufficient for every segment: the reserved channels can be rematched at each boundary without changing their ownership. Hence, Equations 13 and 14 are both necessary and sufficient, and therefore solve Equation 12 optimally.

Algorithm 3 constructs the compact physical pools and binds each segment configuration to them.

When physical resources are managed as bidirectional bundles, each reserved bundle contributes one fixed Tx and one fixed Rx channel. The minimum number of retained bundles is

$$\bar{n}_u^{\text{bi}} = \max \{ \bar{n}_u^{\text{tx}}, \bar{n}_u^{\text{rx}} \}.$$

A bundle cannot be released when either its Tx side or its Rx side is used by any segment. This rule reserves and releases physical attachments conservatively without converting one channel direction into the other.

Algorithm 3 Schedule-Wide Channel Compaction

Require: Integer schedule $\{\mathbf{X}^{(m)}\}_{m=1}^M$ and physical channel inventory

Ensure: Reserved pools, physical matchings, and exposed resources

- 1: **for all** endpoints u **do**
 - 2: Compute $\bar{n}_u^{\text{tx}}$ and $\bar{n}_u^{\text{rx}}$ using Equations 13 and 14
 - 3: Reserve fixed Tx and Rx pools of these sizes
 - 4: **end for**
 - 5: **for** $m = 1$ to M **do**
 - 6: Mark the reserved pools as available
 - 7: **for all** ordered pairs (u, v) **do**
 - 8: Bind $X_{uv}^{(m)}$ distinct Tx channels of u to distinct Rx channels of v
 - 9: **end for**
 - 10: Store the resulting physical matching $\mathcal{M}^{(m)}$
 - 11: **end for**
 - 12: Expose all channels outside the reserved pools
 - 13: **return** the pools, matchings, and exposed resources
-